

Helm Revisited

Q1. what happens in behind the scene when I write these commands helm install, helm upgrade, helm uninstall... Write a sample helm chart and what is the mannual alternative if I were not using helm , I were just using the kubectl commands

1. helm install my-release my-chart

Behind the Scenes:

- Helm reads Chart.yaml and values.yaml.
- It **renders templates** in templates/ using values from values.yaml.
- It generates Kubernetes YAML manifests and sends them to the Kubernetes API via kubectl apply.
- Kubernetes creates the required resources (Deployments, Services, ConfigMaps, etc.).
- Helm stores the release metadata in the **Helm storage (ConfigMap or Secrets in kube-system)**.

Manual Alternative Using kubectl:

```
# Apply Kubernetes YAML manifests manually
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl apply -f configmap.yaml
```

2. helm upgrade my-release my-chart

Behind the Scenes:

- Helm fetches the **existing release** from storage.
- It **merges updated values** from values.yaml with the existing configuration.
- It generates a new Kubernetes YAML manifest.
- It applies kubectl apply to update resources.
- If needed, it restarts the Deployment/Pod based on changes.

Manual Alternative Using kubectl:

```
# Update deployment manually
kubectl set image deployment/my-app my-app-container=my-image:v2
kubectl apply -f updated-deployment.yaml
kubectl apply -f updated-configmap.yaml
```

3. helm uninstall my-release

Behind the Scenes:

- Helm retrieves the release information.
- It runs kubectl delete on all resources in the chart.
- It removes the release metadata from the Helm storage.

Manual Alternative Using kubectl:

```
# Delete all resources manually
kubectl delete -f deployment.yaml
kubectl delete -f service.yaml
kubectl delete -f configmap.yaml
```

Sample Helm Chart

Let's create a simple Helm chart for an Nginx web server.

1. Create a Helm Chart

```
helm create my-nginx-chart
```

This creates a folder my-nginx-chart/ with default files.

2. Modify values.yaml

```
replicaCount: 2
image:
  repository: nginx
  tag: latest
service:
  type: ClusterIP
  port: 80
```

3. Modify templates/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-nginx
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          ports:
            - containerPort: 80
```

4. Modify templates/service.yaml

```
yaml
CopyEdit
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-nginx
spec:
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: {{ .Values.service.port }}
      targetPort: 80
    type: {{ .Values.service.type }}
```

Manual Alternative Without Helm

Instead of using helm install, we would apply raw Kubernetes YAML manifests.

1. Deployment YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

2. Service YAML

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: ClusterIP
```

3. Deploy Using kubectl

```
kubectl apply -f deployment.yaml
```

```
kubectl apply -f service.yaml
```

Q2. What is Release in Helm. Tell me in the context of GKE

A **Release** in Helm is a **deployed instance** of a Helm **chart** in a Kubernetes cluster. When you use `helm install`, Helm creates a **release** by installing the chart with a specific name and configuration.

Each release is:

- **Identified by a unique name** (e.g., `my-app`).
- **Versioned**, allowing upgrades and rollbacks.
- **Stored in Kubernetes** as ConfigMaps or Secrets in the `kube-system` namespace.

Helm Release in the Context of GKE (Google Kubernetes Engine)

When you deploy a Helm chart in **GKE**, a release represents the set of Kubernetes objects (Deployments, Services, ConfigMaps, etc.) deployed within your GKE cluster.

Example: Deploying a Helm Chart in GKE

1. Authenticate with GKE

```
gcloud container clusters get-credentials my-gke-cluster --zone us-central1
```

2. Install a Helm Chart (e.g., Nginx)

```
helm install my-nginx-release my-nginx-chart
```

- This creates a **release** named `my-nginx-release`.
- Helm stores metadata about this release in Kubernetes.

1. List Helm Releases in GKE

```
helm list
```

Output:

NAME CHART	NAMESPACE APP VERSION	REVISION	UPDATED	STATUS
---------------	--------------------------	----------	---------	--------

1. my-nginx-release default 1
nginx-chart 1.21.0

2024-03-06 14:30:00

deployed my-

1. Check Installed Kubernetes Resources

`kubectl get all -l app.kubernetes.io/instance=my-nginx-release`
This shows all resources associated with the Helm release.

2. Upgrade the Release

`helm upgrade my-nginx-release my-nginx-chart --set replicaCount=3`
◦ Helm applies the changes, updating the Deployment in GKE.

1. Rollback a Release

`helm rollback my-nginx-release 1`
◦ Restores the previous version of the release.

1. Uninstall a Release

`helm uninstall my-nginx-release`
◦ Removes the deployment but leaves Persistent Volumes unless specified.

Where is the Helm Release Stored in GKE?

- Helm stores release metadata as **Secrets** in Kubernetes under the kube-system namespace:sh
CopyEdit

`kubectl get secrets -n kube-system | grep my-nginx-release`

- You can inspect the release data:sh

`kubectl get secret sh.helm.release.v1.my-nginx-release.v1 -n kube-system -o yaml`

Q3. version: 0.1.0 appVersion:
"1.0" what are these versions ? Is
kubectl get secrets -n kube-
system | grep my-nginx-release
same as version: 0.1.0 ?

Understanding version and appVersion in Helm

When you see this in Chart.yaml:

version: 0.1.0
appVersion: "1.0"

They mean:

- **version (Chart Version)** → The version of the Helm chart itself (e.g., 0.1.0).

- This follows **semantic versioning** (e.g., major.minor.patch).
- Used to track changes in the **Helm chart template** (not the application).
- Example: If you modify values.yaml or templates but **not the app itself**, you bump this version.
- **appVersion (Application Version)** → The version of the actual application being deployed (e.g., "1.0").
 - Used to specify the **application's version** inside the Helm chart.
 - Has **no effect on Helm's logic** but helps users know which version of the app they are deploying.
 - Example: If you update an Nginx version (nginx:1.21 → nginx:1.22), you update appVersion.

Is kubectl get secrets -n kube-system | grep my-nginx-release the Same as version: 0.1.0?

No, they are **not the same**.

- kubectl get secrets -n kube-system | grep my-nginx-release retrieves Helm release metadata stored in Kubernetes.
- Helm stores release history as **Secrets** (or ConfigMaps in older Helm versions).
- These secrets contain information about **past deployments, revisions, and values used**, not the Chart.yaml version.

Example of Helm Release Secret:

```
kubectl get secret sh.helm.release.v1.my-nginx-release.v1 -n kube-system -o yaml
```

This contains:

- The release name (my-nginx-release).
- The deployed chart version (e.g., 0.1.0).
- The applied values from values.yaml.
- The Kubernetes resources created by Helm.

Item	Meaning
version	Helm chart version (tracks template changes).
appVersion	Application version (tracks actual app release).
Helm Release Secret	Stores release history (not the same as version).
helm list	Shows release info (including app & chart versions).

Check Helm Release History

```
helm history my-app
```

Output:

REVISION	UPDATED	STATUS	CHART	APP VERSION	DESCRIPTION
1	2024-03-06 14:00:00	deployed	my-chart-0.1.0	1.0	Install complete
2	2024-03-06 14:10:00	deployed	my-chart-0.1.0	1.0	Upgrade complete

- **Revision (REVISION) is the Helm Release Version.**
- The **chart version (my-chart-0.1.0) stayed the same.**